PENETRATION TEST REPORT

Anyone Can Cook

A recipe-sharing web application

Prepared for	Team Vortex
Prepared by	Team Sentinel
Team members	Bibas Kandel, Bipul Adhikari, Om Nepal, Sushant Aryal, Suwan Aryal
Dates of testing	October 25, 2026
Report date	October 25, 2026
Course	CSC 489 — Web Application Security, Capstone Milestone 2

1. Executive summary

Team Sentinel assessed Anyone Can Cook, a recipe-sharing web application built by Team Vortex for CSC 489. The application lets people register, post updates and recipes, comment, rate recipes, follow other users, and search. We built it locally from the submitted source and tested every page, form, and API endpoint on October 25, 2026 (about six hours), combining source review with live exploitation.

The application is not safe to run outside a closed test environment. We confirmed six working vulnerabilities, three of them critical. The most serious runs operating-system commands on the host through a feature meant only to check whether a link is reachable. Two more critical issues let an attacker log in as any account with no password: one through SQL injection in the login form, and one through a request header the server trusts without checking. Together they give full control of both the application and the machine it runs on.

Risk summary

Total	Critical	High	Medium	Low	Overall risk
6	3	2	1	0	CRITICAL

Top three issues. (1) Command injection in the link checker runs arbitrary shell commands; we read system files and printed host details. (2) SQL injection in login lets an attacker sign in as any user; we took over `alex_cooks`. (3) A trusted `X-User-Id` header lets any request claim to be any user.

Immediate actions. Before this code runs again: remove the shell call in the link checker, rebuild the login query with parameters, and delete the `X-User-Id` shortcut so identity comes only from the signed session. Each is a small, local change (Section 5).

2. Scope and methodology

Scope. Target: Anyone Can Cook (Flask and SQLite, with a single-page HTML/JS frontend), run from the submitted source at `http://127.0.0.1:5000` (port 5055 during testing), using the SQLite database shipped with the project. Full source was provided, so testing was grey/white-box. Features tested: registration, login, sessions and /api/me, profiles, posts, comments, recipes and ratings, follow and unfollow, search, and the link checker at /api/tools/check-url. Out of scope: the host operating system and network, the Python and Flask runtime, third-party libraries, and denial-of-service testing.

Methodology. We followed the OWASP Web Security Testing Guide and mapped findings to the OWASP Top 10 (2021), working through the feature list one endpoint at a time. For each input we traced how the value reaches the database, the shell, or the HTML response, then sent real requests with curl to confirm the effect: a command running, a login succeeding, a record changing, or data leaking. We captured the request and response for every confirmed issue; those captures are the figures in Section 3.

Tools. curl for crafting and replaying HTTP requests, browser developer tools for the frontend, manual source review of the Flask routes, and the werkzeug library to generate a password hash for the Finding 2 proof.

Accounts. The seeded accounts share the password password123. Several findings need no account, and where a session was required we obtained one through the vulnerabilities themselves rather than by logging in normally.

Cross-site scripting (required category). We tested stored XSS in posts, comments, recipes, and usernames. The frontend passes every user value through an `esc()` escaping helper before rendering, so a payload such as `<img src=x onerror=alert(1)>` is stored but displayed as inert text. Script execution was not possible, so XSS is reported as tested and currently mitigated (Section 4).

Limitation. Testing ran against a local development build; a production configuration could differ. The debug settings in Finding 6 shipped in the submitted code, so we report them as found.

3. Vulnerability findings

Findings are ordered by severity, highest first. Each one gives its location, the cause, steps to reproduce it, a captured proof, the impact, and a specific fix.

3.1 Critical severity

Finding 1: Remote command injection in the link checker

Severity: CRITICAL

Location: GET /api/tools/check-url · parameter url · routes/tools.py lines 22–23 · A03:2021 Injection / CWE-78

Description. The link checker joins a fixed `ping` string with the user's `url` value and runs it with shell=True. The shell interprets characters such as ; && and |, so anything after them runs as a separate command. The value is never validated or quoted, so a request can run any command the server's user can run. The endpoint requires a login, but Finding 3 removes that with a single header.

```
command = "ping -c 1 -W 2 " + target
result = subprocess.run(command, shell=True, capture_output=True, text=True, timeout=5)
```

Reproduction. Send GET /api/tools/check-url with header X-User-Id: 1, and set url to `127.0.0.1; id; uname -a`. The JSON output field returns the result of id and uname -a. Using `0; head -n 5 /etc/passwd` reads a file off the host.

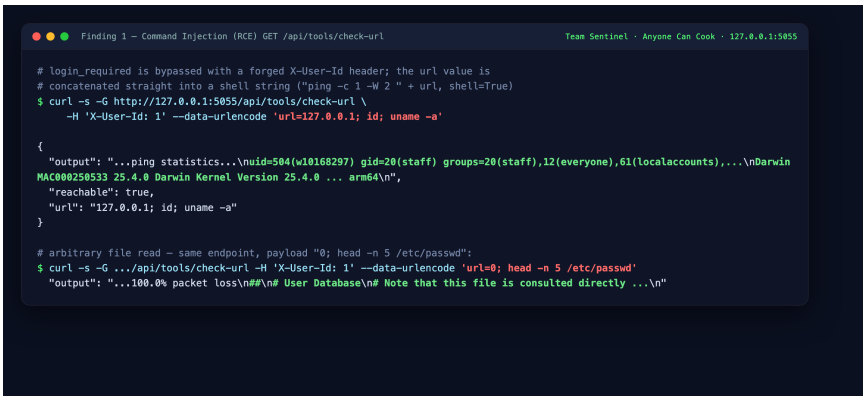


Figure 1. *The url value 127.0.0.1; id; uname -a runs on the server; the response returns uid=504(w10168297) and kernel details, and a second payload reads /etc/passwd.*

Impact. Full command execution as the server's user: read and write files, reach the SQLite database, open a reverse shell, or pivot on the host. This is a complete server compromise affecting all users, and recovery means rebuilding the host.

Remediation. Never build a shell command from user input. Validate the host, then call ping with an argument list and no shell.

```
import socket, subprocess
socket.gethostbyname(host)   # rejects shell metacharacters as invalid hosts
subprocess.run(["ping","-c","1","-W","2",host], capture_output=True, text=True, timeout=5)
```

References: OWASP Command Injection; CWE-78.

Finding 2: SQL injection in login leads to account takeover

Severity: CRITICAL

Location: POST /api/login · parameter username · routes/accounts.py line 41 · A03:2021 Injection / CWE-89

Description. The login route concatenates the username value straight into the SQL statement. An attacker can add a UNION that returns a row of their choosing. The route then checks the submitted password against that row's password value, so supplying a row whose password is an attacker-generated hash makes the check pass and logs the attacker in as any account. Every other query in the project uses parameters; this one does not.

```
sql = "SELECT user_ID, password FROM Users WHERE username = '" + username + "'"
if user and auth.verify_password(user["password"], password):
    auth.login_user(user["user_ID"])
```

Reproduction. Generate a hash with generate_password_hash("pwned123"), then POST /api/login with username = `zzz' UNION SELECT 1, '<hash>' --` and password = pwned123. The response returns alex_cooks (ID 1) and a session cookie; GET /api/me with that cookie confirms the session.

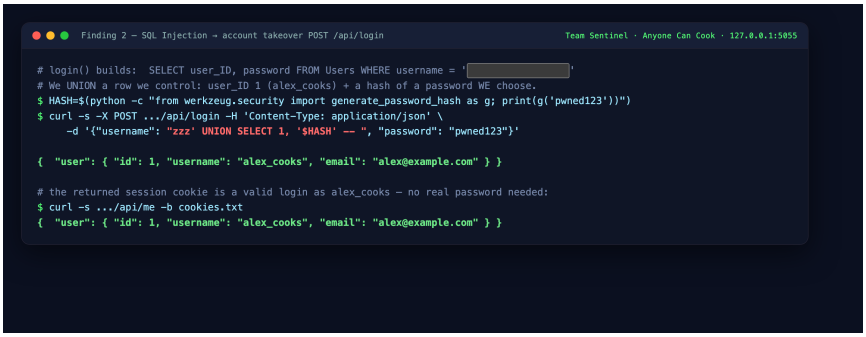


Figure 2. *The UNION payload returns ID 1 with an attacker-chosen hash; the login succeeds and /api/me confirms the session is alex_cooks, with no knowledge of the real password.*

Impact. Authentication bypass and takeover of any account by ID. The same injection point can read other tables through further UNION queries, which exposes the whole user table and its password hashes. Affects all users.

Remediation. Use a parameterized query so the username is treated as data. Also add login rate limiting and lockout.

```
sql = "SELECT user_ID, password FROM Users WHERE username = ?"
user = db.query(sql, (username,), one=True)
```

References: OWASP SQL Injection; CWE-89.

Finding 3: Authentication bypass through a trusted request header

Severity: CRITICAL

Location: header X-User-Id on any authenticated endpoint (e.g. GET /api/me) · auth.py lines 36–40 · A07:2021 / CWE-287, CWE-639

Description. The function that answers "who is calling?" first looks for an X-User-Id request header and, if present, treats its value as the logged-in user with no further check. Any client can set this header to any number, which replaces the entire login and session system with a value the attacker controls. This is what makes Findings 1 and 4 reachable without an account.

```
header_uid = request.headers.get("X-User-Id")
if header_uid is not None:
    return db.query("SELECT ... FROM Users WHERE user_ID = ?", (header_uid,), one=True)
```

Reproduction. GET /api/me with X-User-Id: 1 and no cookie returns alex_cooks; X-User-Id: 3 returns jordan_foodie; a request with no header returns null.

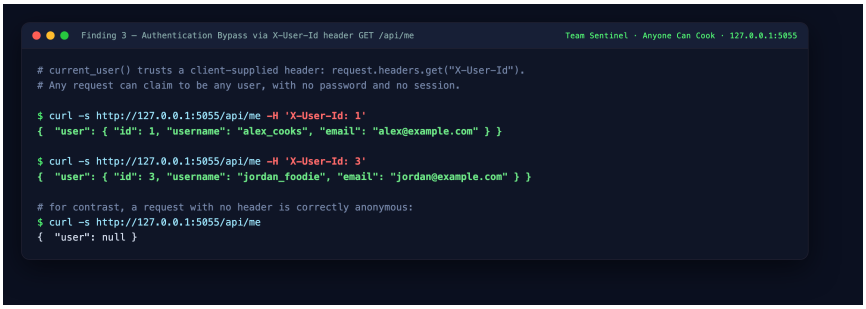


Figure 3. *With no login and no cookie, two header values return two different real accounts, while a request with no header is correctly anonymous.*

Impact. Complete authentication bypass for every protected action by setting one header, including writes. There is effectively no access control. Affects all users.

Remediation. Delete the header branch and take identity only from the signed session cookie.

```
uid = session.get("user_id")
if uid is None: return None
return db.query("SELECT ... FROM Users WHERE user_ID = ?", (uid,), one=True)
```

References: OWASP Broken Authentication; CWE-639.

3.2 High severity

Finding 4: Broken access control lets any user edit or delete others' content

Severity: HIGH

Location: PUT/DELETE /api/posts|recipes|comments/<id> · posts.py (51,65), recipes.py (58,72), comments.py (43); unused helper can_modify() at auth.py:60 · A01:2021 / CWE-639

Description. The edit and delete routes check that the caller is logged in, but never that the caller owns the item. The project even defines can_modify(owner_id) for this, but no route calls it. Any logged-in user can change or remove any post, recipe, or comment by its ID. Paired with Finding 3, no real account is needed.

```
# routes/posts.py - update_post(): owner is read but never checked
post = db.query("SELECT user_ID FROM Posts WHERE post_ID = ?", (post_id,), one=True)
db.execute("UPDATE Posts SET content = ? WHERE post_ID = ?", (content, post_id))
```

Reproduction. Post 5 belongs to morgan_eats (user 5). Send PUT /api/posts/5 with X-User-Id: 2 and new content; the response returns post 5 with the new text under morgan_eats's name.



Figure 4. *Acting as user 2, we overwrote post 5 (owned by user 5); the before and after lines show the content changed under another user's name.*

Impact. Any user can tamper with or destroy any other user's content, including a mass wipe by iterating over IDs. This breaks data integrity for everyone who creates content.

Remediation. Check ownership on every edit and delete, using the helper that already exists.

```
if not auth.can_modify(post["user_ID"]):
    return error("Forbidden", 403)
```

References: OWASP Broken Access Control; CWE-639.

Finding 5: Password hashes and emails exposed on the public profile endpoint

Severity: HIGH

Location: GET /api/users/<id> (no authentication) · routes/accounts.py line 64 · A02:2021 Cryptographic Failures / CWE-200

Description. The public profile query selects the password column along with the display fields and returns the whole row as JSON. The endpoint needs no login, so anyone can read any user's stored password hash and email address by walking through the ID numbers.

```
# routes/accounts.py - profile(): 'password' is returned to the client
"SELECT user_ID AS id, username, email, password, acc_date_created AS created ..."
```

Reproduction. GET /api/users/1 with no authentication returns profile.password (the full scrypt hash). Repeat for IDs 2, 3, and so on to collect every hash and email.

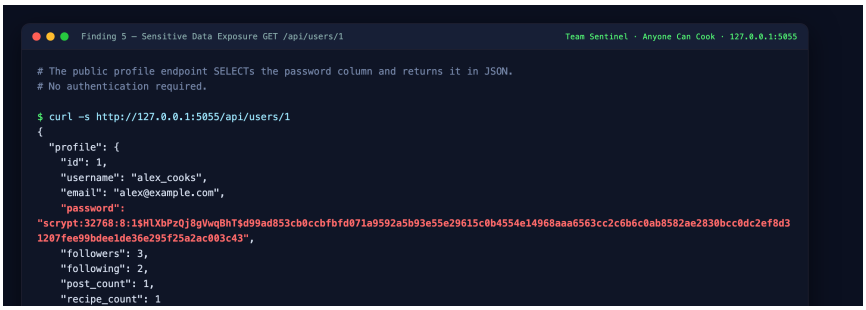


Figure 5. An unauthenticated request to /api/users/1 returns the account's full scrypt password hash and email address in the JSON body.

Impact. Every hash is readable without logging in and can be cracked offline; weak or reused passwords fall quickly, and the exposure can reach other sites. Emails support phishing and account enumeration. Affects all users.

Remediation. Never select or return the password column for display; list the fields explicitly and leave it out.

```
"SELECT user_ID AS id, username, email, acc_date_created AS created FROM Users WHERE
user_ID = ?"
```

References: OWASP Cryptographic Failures; CWE-200.

3.3 Medium severity

Finding 6: Debug mode and a hardcoded secret key in shipped code

Severity: MEDIUM

Location: config.py line 8 (SECRET_KEY) and line 10 (DEBUG = True) · application-wide · A05:2021 / CWE-489, CWE-798, CWE-215

Description. The application ships with DEBUG = True and a fixed SECRET_KEY of "secret123". With debug on, any unhandled error returns the Werkzeug interactive debugger, which shows the stack trace, source code, and file paths, and can allow code execution through its console. The hardcoded key is public in the source, so anyone can forge valid session cookies for any user.

```
SECRET_KEY = "secret123"
DEBUG = True
```

Reproduction. Send a request that triggers a server error, for example a login with an unbalanced quote in the username (which also hits Finding 2). The HTTP 500 response is the Werkzeug debugger page, exposing the traceback, source lines, and absolute file paths.



Figure 6. *A malformed login returns the Werkzeug debugger with a full traceback, the sqlite3.OperationalError message, source code, and absolute host paths.*

Impact. Error pages leak source, paths, and library versions that help an attacker plan further attacks; the known key allows forged session cookies (a second route to Finding 3); and the debugger console can allow code execution if reachable. Affects all users and the host.

Remediation. Default debug off and load a random secret from the environment, not from source.

```
import os
SECRET_KEY = os.environ["ACC_SECRET_KEY"]   # long random value per environment
DEBUG = os.environ.get("ACC_DEBUG") == "1"  # default off
```

References: OWASP Security Misconfiguration; CWE-489.

3.4 Low severity

No low-severity vulnerabilities were confirmed. Cross-site scripting, a required test category, was attempted against posts, comments, recipes, and usernames and was found mitigated by consistent output escaping in the frontend (see Sections 2 and 4).

4. Summary of findings

#	Vulnerability	Severity	Location	Status
1	Command injection (RCE)	Critical	/api/tools/check-url	Open
2	SQL injection in login	Critical	/api/login	Open
3	Auth bypass via X-User-Id	Critical	auth.py current_user()	Open
4	Broken access control (IDOR)	High	/api/posts|recipes|comments	Open

#	Vulnerability	Severity	Location	Status
5	Password hash + email exposure	High	/api/users/<id>	Open
6	Debug mode + hardcoded secret	Medium	config.py	Open

4.2 Risk assessment

Overall posture: Critical. Three separate critical issues each hand an attacker control of accounts or the host, and they reinforce each other: the trusted header removes the login requirement that would slow Findings 1 and 4, and the data exposure in Finding 5 feeds the hash cracking that Finding 2 also enables. The common root cause is trusting user input at the point where it reaches a shell, a SQL statement, or an identity decision.

Controls done well. The frontend escapes all user-generated content, which closed off stored XSS in every field tested. Passwords are stored with werkzeug scrypt hashing rather than plain text, so Finding 5 leaks hashes rather than raw passwords. Session cookies set HttpOnly and SameSite=Lax, which blocks basic cross-site request forgery. Most database queries use parameters correctly; the login query is the exception.

Gaps. User input reaches a shell and a SQL query without sanitization; authorization is missing on content changes and is overridden by a client header; a sensitive column is returned to anonymous callers; debug configuration and a public secret key shipped in the code; and there is no rate limiting on login.

5. Recommendations

5.1 Immediate (next 48 hours)

- Remove the shell call in the link checker (Finding 1): call ping with an argument list and validate the host. About 1 hour.
- Parameterize the login query (Finding 2). About 15 minutes.
- Delete the X-User-Id branch (Finding 3) and read identity from the session. About 30 minutes.

5.2 Short-term (next week)

- Enforce ownership on edit and delete (Finding 4) with the existing can_modify() helper.
- Stop returning the password column (Finding 5) and limit email to the account owner.
- Default debug off and load the secret key from the environment (Finding 6).
- Add login rate limiting, lockout, and logging of failed attempts.

5.3 Long-term

- Adopt a rule that no SQL or shell command is built by string concatenation; use parameterized queries and argument-list subprocess calls only.
- Centralize authorization so every state-changing route passes an ownership or role check.
- Run the Bandit linter in the build; it flags shell=True and the debug setting automatically.
- Add an input, auth, and configuration review checklist before merging.

5.4 Remediation roadmap

Window	Work
Days 1–2	Fix the three critical findings (1, 2, 3); redeploy the test build.
Week 1	Fix the high findings (4, 5), correct the configuration (6), add login throttling.
Ongoing	Add the Bandit linter and review checklist; retest after each change.

6. Appendix

A. Payloads used

Category	Payload
Command injection	127.0.0.1; id; uname -a / 0; head -n 5 /etc/passwd
SQL injection (login)	zzz' UNION SELECT 1, '<attacker-hash>' --
Auth bypass	header X-User-Id: 1 (and : 3)
Broken access control	PUT /api/posts/5 with X-User-Id: 2
Data exposure	GET /api/users/1 (no auth)
Misconfiguration	{"username":"broken'quote","password":"x"} (debug traceback)
XSS (mitigated)	<img src=x onerror=alert(1)> stored, rendered inert

B. Testing environment and references

Target run from source in a local virtual environment (Flask 3.1, Python 3.12) bound to 127.0.0.1:5055, using the SQLite database shipped with the project. Client tools: curl, a browser, and werkzeug. References: OWASP Top 10 (2021) at owasp.org/Top10; OWASP Web Security Testing Guide; CWE-78, CWE-89, CWE-287, CWE-639, CWE-200, and CWE-489 at cwe.mitre.org.